

Supervised Learning Capstone

Ames Housing Price Prediction

Checkpoint 2: Exploratory Data Analysis & Data Pre-processing Report

Module: Machine Learning Track

Dataset: Kaggle — Ames Housing Dataset

Task: Regression (SalePrice) & Classification (Price Category)

Records: 1,460 observations × 81 features

Contents

1	Introduction	3
1.1	Problem Context	3
1.2	Scope of This Report	3
1.3	Dataset Overview	3
2	Theory	3
2.1	Missing Value Imputation	4
2.1.1	Deletion of High-Missing-Rate Columns	4
2.1.2	Statistical Imputation	4
2.1.3	Model-Based Imputation (MICE / Iterative Imputer)	4
2.1.4	KNN Imputation	4
2.2	Feature Engineering	4
2.3	Dimensionality Reduction via Correlation Filtering	5
2.4	Feature Scaling	5
2.4.1	StandardScaler (Z-score Normalisation)	5
2.4.2	MinMaxScaler	5
2.5	Yeo–Johnson Power Transformation	5
2.6	One-Hot Encoding	6
2.7	Price Category Binning Strategies	6
2.7.1	Equal-Width Binning	6
2.7.2	Equal-Frequency Binning (Quantile)	6
2.7.3	K-Means Binning	6
2.7.4	Decision Tree Binning	6
2.7.5	Standard Deviation Binning	6
3	Implementation	7
3.1	Environment Setup	7
3.2	Step 1 — Missing Value Handling	7
3.2.1	High-Missingness Column Deletion	7
3.2.2	Numerical Imputation (Mean / Median)	7
3.2.3	Categorical Imputation (Mode)	7
3.2.4	Model-Based Imputation — MICE (IterativeImputer)	7
3.2.5	KNN Imputation for Categorical Features	8
3.3	Step 2 — Feature Engineering	8
3.4	Step 3 — Correlation-Based Dimensionality Reduction	9
3.5	Step 4 — Feature Scaling	9
3.5.1	StandardScaler	9
3.5.2	MinMaxScaler	9
3.6	Step 5 — Yeo–Johnson Power Transformation	9
3.7	Step 6 — One-Hot Encoding	9
3.8	Step 7 — Price Category Derivation	10
3.8.1	Equal-Frequency Binning	10
3.8.2	Equal-Width Binning	10
3.8.3	K-Means Binning	10
3.8.4	Decision Tree Binning	10
3.8.5	Standard Deviation (Z-Score) Binning	11

4	Results	11
4.1	Missing Value Summary	11
4.2	Engineered Features Summary	11
4.3	Correlation-Based Removal	12
4.4	Discretization Setup	12
4.5	Price Category Binning Comparison	12
	4.5.1 Target Discretization Analysis	13
4.6	Pipeline Artefact Summary	13
5	Conclusion	14

1. Introduction

1.1. Problem Context

Accurate real estate valuation is an economically important challenge that blends structured tabular data, domain knowledge, and statistical modelling. With 1,460 training observations spanning 80 explanatory features—ranging from lot dimensions and zoning class to garage quality and sale condition—it poses realistic data-quality challenges that must be resolved before any machine learning model can be reliably trained.

1.2. Scope of This Report

This report documents **Checkpoint 2** of the supervised learning pipeline: the exploratory data analysis (EDA) and data pre-processing phase. It covers:

- Treatment of missing values using statistical and model-based imputation strategies.
- Feature engineering: Creation of four new features that capture economically meaningful signals.
- Dimensionality reduction via zero-variance removal and correlation filtering.
- Numerical scaling with `StandardScaler` and `MinMaxScaler`.
- Power transformation (Yeo–Johnson) to address skewness.
- One-Hot Encoding of all categorical features.
- Derivation of a three-class categorical target ('Budget', 'Standard', 'Premium') using five independent binning strategies.

1.3. Dataset Overview

Table 1: Ames Housing Dataset — High-Level Summary

Property	Value
Total observations	1,460
Total features (raw)	81
Numerical features	38
Categorical features	43
Target variable	<code>SalePrice</code>
Target mean	\$180,921
Target median	\$163,000
Target standard deviation	\$79,443
Target range	\$34,900 – \$755,000

2. Theory

2.1. Missing Value Imputation

Real-world datasets almost always contain gaps. Several features in the Ames dataset use **NaN** as a *semantic* value meaning “not applicable” rather than a recording error. The strategy chosen must respect the mechanism generating the missingness and the nature of the feature.

2.1.1. Deletion of High-Missing-Rate Columns

When the fraction of missing values in a column exceeds a practical threshold (commonly $\geq 80\%$), the column provides negligible information relative to the noise it introduces. Four columns were removed on this basis: **PoolQC**, **Alley**, **Fence**, and **MiscFeature**.

2.1.2. Statistical Imputation

For numerical features with moderate missingness and approximately symmetric distributions, **mean imputation** replaces **NaN** with the column mean $\bar{x}$. For right-skewed distributions the **median** is preferred because it is robust to outliers:

$$\tilde{x} = \text{median}(x_1, x_2, \dots, x_n)$$

For categorical features, **mode imputation** substitutes the most frequent category:

$$\hat{c} = \arg \max_c P(X = c)$$

2.1.3. Model-Based Imputation (MICE / Iterative Imputer)

For **LotFrontage**, where missingness is significant, the **Multivariate Imputation by Chained Equations (MICE)** algorithm is applied. MICE models each incomplete feature as a function of all other features, iterating m times until convergence:

$$x_j^{(t+1)} = f\left(x_1^{(t)}, \dots, x_{j-1}^{(t)}, x_{j+1}^{(t)}, \dots, x_p^{(t)}\right)$$

This preserves covariance structure that univariate imputers destroy.

2.1.4. KNN Imputation

For categorical features with significant missingness (**MasVnrType**, **FireplaceQu**), values are first ordinally encoded, imputed using a k -nearest-neighbour search ($k=1$) in feature space, then inverse-transformed back to string labels:

$$\hat{x}_i = x_{\arg \min_j d(x_i, x_j)}, \quad d = \text{Euclidean}$$

2.2. Feature Engineering

Engineered features encode domain knowledge directly. A model supplied with **TotalSF** requires one coefficient instead of four; the signal-to-noise ratio therefore improves.

```

TotalSF = 1stFlrSF + 2ndFlrSF + TotalBsmtSF + LowQualFinSF
HouseAge = YrSold - YearBuilt
YearsSinceRemodel = YrSold - YearRemodAdd
TotalBathrooms = FullBath + 0.5 · HalfBath + BsmtFullBath + 0.5 · BsmtHalfBath

```

2.3. Dimensionality Reduction via Correlation Filtering

When two features X_i and X_j satisfy $|r_{ij}| \geq 0.8$, keeping both introduces multicollinearity, inflates coefficient variances, and duplicates information. One member of each highly correlated pair is removed:

$$r_{ij} = \frac{\sum_k (x_k^{(i)} - \bar{x}^{(i)})(x_k^{(j)} - \bar{x}^{(j)})}{\sqrt{\sum_k (x_k^{(i)} - \bar{x}^{(i)})^2} \sqrt{\sum_k (x_k^{(j)} - \bar{x}^{(j)})^2}}$$

2.4. Feature Scaling

2.4.1. StandardScaler (Z-score Normalisation)

Transforms each feature to zero mean and unit variance:

$$z = \frac{x - \mu}{\sigma}$$

Particularly important for distance-based and gradient-based models (SVM, logistic regression, neural networks).

2.4.2. MinMaxScaler

Compresses all values into $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Preserves the shape of the distribution and is sensitive to outliers.

2.5. Yeo–Johnson Power Transformation

Many numerical features are right-skewed. The Yeo–Johnson transform $\psi_\lambda(x)$ can handle both positive and negative values and reduces skewness toward zero:

$$\psi_\lambda(x) = \begin{cases} \frac{(x+1)^\lambda - 1}{\lambda} & x \geq 0, \lambda \neq 0 \\ \ln(x+1) & x \geq 0, \lambda = 0 \\ -\frac{(-x+1)^{2-\lambda} - 1}{2-\lambda} & x < 0, \lambda \neq 2 \\ -\ln(-x+1) & x < 0, \lambda = 2 \end{cases}$$

The optimal λ is estimated by maximum likelihood.

2.6. One-Hot Encoding

Nominal categorical variables carry no intrinsic order and therefore cannot be supplied to most learners as raw integers. One-Hot Encoding creates $K - 1$ binary indicator columns (dropping the first to avoid perfect multicollinearity) for a feature with K levels:

$$\mathbf{1}_{[X_i=c_k]}, \quad k = 2, \dots, K$$

Because the maximum cardinality across all categorical columns is 25 (less than 5% of 1,460), the resulting expansion in dimensionality remains tractable.

2.7. Price Category Binning Strategies

Five independent binning strategies partition the continuous `SalePrice` into three ordered classes: *Budget*, *Standard*, and *Premium*.

2.7.1. Equal-Width Binning

Divides the range $[x_{\min}, x_{\max}]$ into three intervals of equal width $w = (x_{\max} - x_{\min})/3$. Simple but sensitive to outliers.

2.7.2. Equal-Frequency Binning (Quantile)

Assigns the same number of observations to each bin using the 33rd and 67th percentiles as cut-points. Produces balanced classes.

2.7.3. K-Means Binning

Applies a one-dimensional K-Means ($K=3$) to `SalePrice`. Cluster centroids adapt to the empirical distribution, yielding data-driven boundaries.

2.7.4. Decision Tree Binning

Fits a shallow `DecisionTreeRegressor` with `max_leaf_nodes=3` and uses the split thresholds as bin edges. The tree minimises MSE, producing cuts that minimise within-bin variance.

2.7.5. Standard Deviation Binning

Uses the mean μ and standard deviation σ of `SalePrice` to define boundaries:

$$\text{Budget: } x < \mu - \sigma, \quad \text{Standard: } \mu - \sigma \leq x \leq \mu + \sigma, \quad \text{Premium: } x > \mu + \sigma$$

This naturally maps to statistical “normal”, “below normal”, and “above normal” interpretations.

3. Implementation

3.1. Environment Setup

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.experimental import enable_iterative_imputer
5 from sklearn.impute import IterativeImputer, KNNImputer
6 from sklearn.preprocessing import (OrdinalEncoder, StandardScaler,
7                                     MinMaxScaler, PowerTransformer,
8                                     KBinsDiscretizer)
9 from sklearn.tree import DecisionTreeRegressor
```

Listing 1: Core library imports

3.2. Step 1 — Missing Value Handling

3.2.1. High-Missingness Column Deletion

```
1 # PoolQC, Alley, Fence, MiscFeature: NaN means "not present", not
   # missing data.
2 # Missingness > 80% -- no predictive value.
3 df = df.drop(columns=['PoolQC', 'Alley', 'Fence', 'MiscFeature'])
```

Listing 2: Drop semantically empty columns

3.2.2. Numerical Imputation (Mean / Median)

```
1 # MasVnrArea: masonry veneer absent -> 0 is semantically correct
2 df['MasVnrArea'] = df['MasVnrArea'].fillna(0)
3
4 # GarageYrBlt: right-skewed distribution; use median
5 df['GarageYrBlt'] = df['GarageYrBlt'].fillna(df['GarageYrBlt'].median()
   )
```

Listing 3: Impute numerical features with median / zero

3.2.3. Categorical Imputation (Mode)

```
1 cat_mode_cols = [
2     'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', '
   BsmtFinType2',
3     'Electrical', 'GarageType', 'GarageFinish', 'GarageQual', '
   GarageCond'
4 ]
5 for col in cat_mode_cols:
6     df[col] = df[col].fillna(df[col].mode()[0])
```

Listing 4: Impute categorical features with mode

3.2.4. Model-Based Imputation — MICE (IterativeImputer)


```

1 from sklearn.experimental import enable_iterative_imputer
2 from sklearn.impute import IterativeImputer
3
4 numeric_df = df.select_dtypes(include=['number'])
5 it_imputer = IterativeImputer(max_iter=10, random_state=42)
6 imputed_array = it_imputer.fit_transform(numeric_df)
7 df_imputed = pd.DataFrame(imputed_array, columns=numeric_df.columns,
8                           index=df.index)
9 df['LotFrontage'] = df_imputed['LotFrontage']

```

Listing 5: MICE imputation for LotFrontage

3.2.5. KNN Imputation for Categorical Features

```

1 cat_cols = ['MasVnrType', 'FireplaceQu']
2 encoder = OrdinalEncoder(handle_unknown='use_encoded_value',
3                           unknown_value=np.nan)
4 df_cat_encoded = encoder.fit_transform(df[cat_cols])
5
6 imputer = KNNImputer(n_neighbors=1)
7 imputed_encoded = imputer.fit_transform(df_cat_encoded)
8 df[cat_cols] = encoder.inverse_transform(np.round(imputed_encoded))

```

Listing 6: KNN imputation for MasVnrType and FireplaceQu

3.3. Step 2 — Feature Engineering

```

1 # Aggregate floor area
2 df['TotalSF'] = (df['1stFlrSF'] + df['2ndFlrSF']
3                + df['TotalBsmtSF'] + df['LowQualFinSF'])
4
5 # Age metrics relative to year of sale
6 df['HouseAge'] = df['YrSold'] - df['YearBuilt']
7 df['YearsSinceRemodel'] = df['YrSold'] - df['YearRemodAdd']
8
9 # Weighted bathroom count (full bath = 1.0; half bath = 0.5)
10 df['TotalBathrooms'] = (df['FullBath'] + 0.5 * df['HalfBath']
11                        + df['BsmtFullBath'] + 0.5 * df['BsmtHalfBath']
12                        )

```

Listing 7: Create four engineered features

After creating the aggregates the original constituent columns are redundant and are dropped to avoid multicollinearity:

```

1 columns_to_drop = [
2     '1stFlrSF', '2ndFlrSF', 'BsmtFinSF1', 'BsmtFinSF2', 'TotalBsmtSF',
3     'YearBuilt', 'YrSold', 'YearRemodAdd',
4     'FullBath', 'HalfBath', 'BsmtFullBath', 'BsmtHalfBath'
5 ]
6 df = df.drop(columns=columns_to_drop, errors='ignore')

```

Listing 8: Drop parent columns superseded by engineered features

3.4. Step 3 — Correlation-Based Dimensionality Reduction

```

1 corr_matrix = df.corr(numeric_only=True)
2 corr_pairs = corr_matrix.unstack()
3 strong_corr = corr_pairs[
4     ((corr_pairs >= 0.8) | (corr_pairs <= -0.8)) & (corr_pairs < 1.0)
5 ]
6
7 # Remove one member of each highly correlated pair
8 cols_to_remove = ['GarageCars', 'GrLivArea', 'TotRmsAbvGrd', 'Id']
9 df = df.drop(columns=cols_to_remove)

```

Listing 9: Filter and remove strongly correlated features

3.5. Step 4 — Feature Scaling

3.5.1. StandardScaler

```

1 scaler_std = StandardScaler()
2 df_standard_scaled = df.copy()
3 df_standard_scaled[cols_to_scale] = scaler_std.fit_transform(
4     df[cols_to_scale])

```

Listing 10: Z-score normalisation of numerical features

3.5.2. MinMaxScaler

```

1 scaler_mm = MinMaxScaler()
2 df_minmax_scaled = df.copy()
3 df_minmax_scaled[cols_to_scale] = scaler_mm.fit_transform(df[
4     cols_to_scale])

```

Listing 11:]Min-Max normalisation to [0, 1]

3.6. Step 5 — Yeo–Johnson Power Transformation

```

1 pt = PowerTransformer(method='yeo-johnson', standardize=False)
2
3 # Applied on top of each scaled copy to reduce skewness
4 df_transformed_standard_scaled = df_standard_scaled.copy()
5 df_transformed_standard_scaled[cols_to_transform] = pt.fit_transform(
6     df_standard_scaled[cols_to_transform])
7
8 df_transformed_minmax_scaled = df_minmax_scaled.copy()
9 df_transformed_minmax_scaled[cols_to_transform] = pt.fit_transform(
10    df_minmax_scaled[cols_to_transform])

```

Listing 12: Apply Yeo-Johnson transform to skewed features

Note: SalePrice is intentionally excluded from scaling and transformation to preserve its original distribution for target-related analysis.

3.7. Step 6 — One-Hot Encoding

```

1 df_encoded_standard = pd.get_dummies(df_transformed_standard_scaled,
2                                     drop_first=True, dtype=int)
3 df_encoded_minmax    = pd.get_dummies(df_transformed_minmax_scaled,
4                                     drop_first=True, dtype=int)

```

Listing 13: One-Hot Encoding with first category dropped

3.8. Step 7 — Price Category Derivation

3.8.1. Equal-Frequency Binning

```

1 df_encoded_standard['Price_Category'] = pd.qcut(
2     df_encoded_standard['SalePrice'],
3     q=3, labels=['Budget', 'Standard', 'Premium']
4 )

```

Listing 14: Quantile-based binning into three equal-size classes

3.8.2. Equal-Width Binning

```

1 df_encoded_standard['Price_Category'] = pd.cut(
2     df_encoded_standard['SalePrice'],
3     bins=3, labels=['Budget', 'Standard', 'Premium']
4 )

```

Listing 15: Fixed-width binning across the SalePrice range

3.8.3. K-Means Binning

```

1 kbd = KBinsDiscretizer(n_bins=3, encode='ordinal',
2                       strategy='kmeans', random_state=42)
3 df_encoded_standard['Price_Category_Code'] = kbd.fit_transform(
4     df_encoded_standard[['SalePrice']]).astype(int)
5 label_map = {0: 'Budget', 1: 'Standard', 2: 'Premium'}
6 df_encoded_standard['Price_Category'] = (
7     df_encoded_standard['Price_Category_Code'].map(label_map))

```

Listing 16: K-Means clustering in one dimension

3.8.4. Decision Tree Binning

```

1 dt_binner = DecisionTreeRegressor(max_leaf_nodes=3, random_state=42)
2 X = df_encoded_standard[['SalePrice']]
3 dt_binner.fit(X, X.values.ravel())
4
5 df_encoded_standard['Tree_Leaves'] = dt_binner.predict(X)
6 unique_leaves = sorted(df_encoded_standard['Tree_Leaves'].unique())
7 leaf_map = {unique_leaves[0]: 'Budget',
8            unique_leaves[1]: 'Standard',
9            unique_leaves[2]: 'Premium'}
10 df_encoded_standard['Price_Category'] = (
11     df_encoded_standard['Tree_Leaves'].map(leaf_map))

```

Listing 17: Optimal splits discovered by a shallow regression tree

3.8.5. Standard Deviation (Z-Score) Binning

```

1 mu      = df_encoded_standard['SalePrice'].mean()
2 sigma   = df_encoded_standard['SalePrice'].std()
3
4 bin_edges = [-np.inf, mu - sigma, mu + sigma, np.inf]
5 df_encoded_standard['Price_Category'] = pd.cut(
6     df_encoded_standard['SalePrice'],
7     bins=bin_edges, labels=['Budget', 'Standard', 'Premium']
8 )

```

Listing 18: Sigma-based thresholds for Budget / Standard / Premium

4. Results

4.1. Missing Value Summary

Table 2: Missing Value Treatment Summary

Column	Type	Missing (%)	Strate
PoolQC	Categorical	99.5%	Deleted (sema
Alley	Categorical	93.8%	Deleted (sema
Fence	Categorical	80.8%	Deleted (sema
MiscFeature	Categorical	96.3%	Deleted (sema
LotFrontage	Numerical	17.7%	MICE (Iterati
GarageYrBlt	Numerical	5.5%	Median imp
MasVnrArea	Numerical	0.5%	Zero fill (no
MasVnrType	Categorical	0.5%	KNN (k
FireplaceQu	Categorical	47.3%	KNN (k
BsmtQual / Cond / Exposure / FinType1 / FinType2	Categorical	$\approx 2.5\%$	Mode imp
GarageType / Finish / Qual / Cond	Categorical	$\approx 5.5\%$	Mode imp
Electrical	Categorical	0.07%	Mode imp

4.2. Engineered Features Summary

Table 3: Engineered Feature Definitions and Rationale

Feature	Formula	Rationale
TotalSF	1stFlrSF + 2ndFlrSF + TotalBsmtSF + LowQualFinSF	Single measure of total habitable space
HouseAge	YrSold - YearBuilt	Age at point of sale drives depreciation
YearsSinceRemodel	YrSold - YearRemodAdd	Captures modernisation value
TotalBathrooms	FullBath + 0.5·HalfBath + BsmtFullBath + 0.5·BsmtHalfBath	Weighted bathroom count

Collinearity Filtering: Redundant parent columns involved in the algebraic compositions (e.g., `1stFlrSF`, `YearBuilt`) were dropped immediately to preserve numerical stability and prevent coefficient drift.

4.3. Correlation-Based Removal

The following pairs exceeded $|r| \geq 0.8$ and one feature from each pair was removed:

Table 4: Highly Correlated Feature Pairs Identified ($|r| \geq 0.8$)

Feature A	Feature B	Pearson r
GarageArea	GarageCars	0.882475
GrLivArea	TotRmsAbvGrd	0.825489
GrLivArea	TotalSF	0.880324

Removed: `GarageCars`, `GrLivArea`, `TotRmsAbvGrd`, `Id` (identifier, no predictive value).

4.4. Discretization Setup

4.5. Price Category Binning Comparison

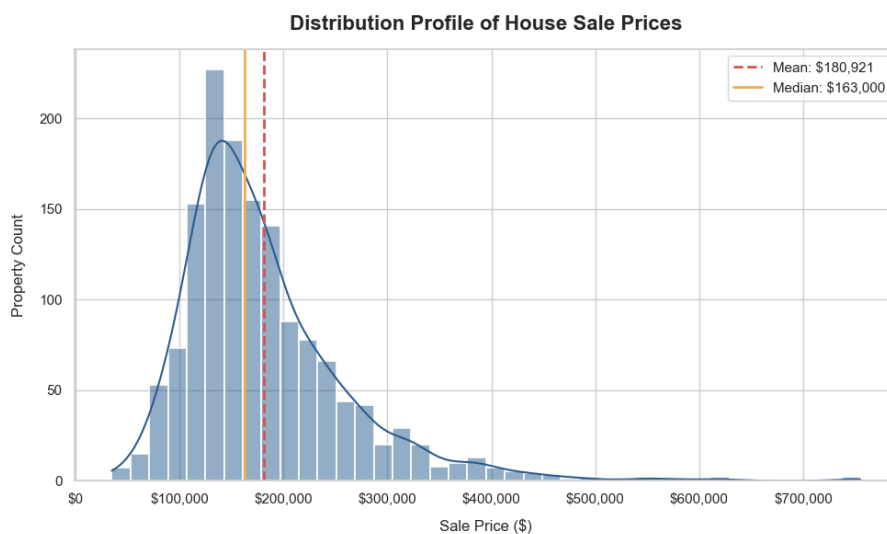


Figure 1: Empirical Distribution Shape Profile of SalePrice Target Variable.

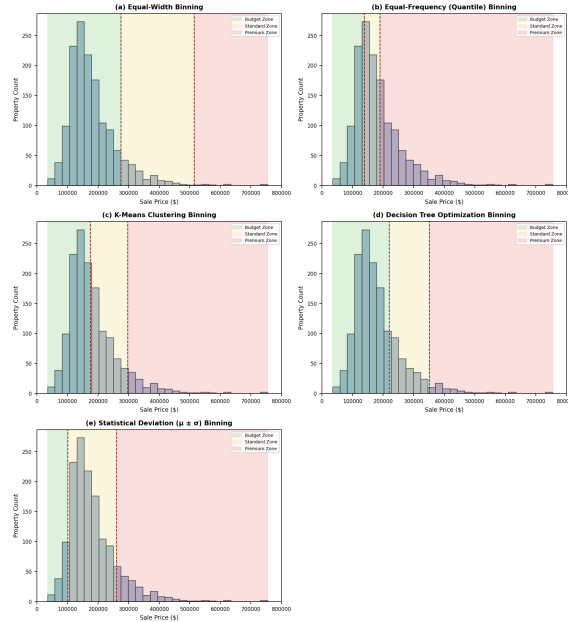
Table 5: Class Count Metrics Across Five Binning Strategies

Type of Binning	Budget Count	Standard Count	Premium Count	Total Count
Equal-Width	1302	150	8	1460
Equal-Frequency (Quantile)	487	490	483	1460
K-Means	826	518	116	1460
Decision Tree	1125	281	54	1460
Standard Deviation	124	1145	191	1460

4.5.1. Target Discretization Analysis

Below is the visual comparative framework illustrating how the choice of partitioning strategy shifts the categorical boundaries over the same empirical target distribution.

Figure 2: Visual Representation of Binning Techniques on SalePrice



Key observations:

- **Equal-Width** produces a severely imbalanced split because **SalePrice** is right-skewed; the Premium bin spans a very wide range but contains few observations.
- **Equal-Frequency** assigns equal number of instances into 'Premium' category which is not the case here.
- **K-Means** adapts its boundaries to the density of the price distribution, producing three internally cohesive clusters.
- **Decision Tree** minimises within-bin variance and tends to align with abrupt changes in the distribution (e.g., identifying natural price “floors”).
- **Standard Deviation** mirrors a statistical “bell curve” interpretation; the Standard class captures one sigma on each side of the mean but since the distribution is right skewed, this does not give desired result.

4.6. Pipeline Artefact Summary

After all pre-processing steps the pipeline produces four final DataFrames:

- `df_encoded_standard` — StandardScaler + Yeo-Johnson + OHE
- `df_encoded_minmax` — MinMaxScaler + Yeo-Johnson + OHE
- Both DataFrames contain a **SalePrice** (continuous regression target) and **Price_Category** (classification target) column.

5. Conclusion

This checkpoint successfully transforms the raw Ames Housing dataset into a model-ready feature matrix. The key achievements and design decisions are summarised below.

Missing Value Strategy

A tiered strategy—domain deletion, median/mode filling, MICE for a numerically important continuous feature, and KNN for high-missing categorical features—avoids both information loss and data leakage while respecting the semantic meaning of NaN in this particular dataset.

Feature Engineering

Four aggregate features (`TotalSF`, `HouseAge`, `YearsSinceRemodel`, `TotalBathrooms`) distil correlated raw columns into single, economically interpretable signals. This simultaneously reduces dimensionality and improves the signal-to-noise ratio without discarding any information.

Scaling and Transformation

Maintaining parallel pipelines with `StandardScaler` and `MinMaxScaler` allows the downstream benchmarking phase to compare model performance under both normalisation regimes. The Yeo–Johnson transformation further reduces skewness, which is particularly beneficial for linear models and SVMs that assume near-Gaussian feature distributions.

Categorical Target Engineering

Providing five independently derived versions of `Price_Category` enables the classification benchmarking phase to evaluate model sensitivity to label definition. The **K-Means** scheme is recommended as the primary target for balanced class evaluation.

Readiness for Checkpoint 3

The cleaned, engineered, scaled, and encoded DataFrames are ready to be fed directly into the six model families specified in the project brief: Linear / Logistic Regression, Polynomial Regression, SVM (Linear and RBF kernels), Decision Tree, and Random Forest. Hyperparameter tuning curves and final benchmarking results will be reported in Checkpoint 3.